Web Architecture and Web Development Technologies

CS406.3

Coursework 2025/2026

Name	**N.W.P.P.P.Nanayakkara**
Index	**29366**
Degree	**BSc (Hons) in Computer Systems Engineering**

Contents

Table of Figures

Introduction & Scenario Analysis

The Sri Lanka Bureau of Foreign Employment (SLBFE) requires a modern web API to facilitate human resource management for the overseas market. The system aims to automate registration, qualification tracking, foreign employment monitoring, and complaint handling. This project delivers a RESTful API and a mobile client that satisfies all functional requirements and scenario items (i) to (vi).

i. Any citizen can become a member through a free online registration.
ii. The citizens who seek jobs, must be able to update their qualifications and upload their birth certificates, CVs, copies of the passports through the system.
iii. The bureau officers must be able to see and validate information provided by the job seekers.
iv. The foreign companies should be able to find workers based on the qualifications.
v. The citizens who have gone for foreign employment must update their current location as soon as they visit the foreign company.
vi. Any citizen can make a complaint and the bureau officers should be able to see the content and reply accordingly

Tools & Technologies

- **Backend Framework**: Node.js with Express.js. Selected for its non-blocking I/O and ability to deliver data in high-speed JSON format.
- **Database:** MongoDB Atlas. Chosen because its document-based structure allows for flexible "Member" profiles as described in the scenario.
- **Security:** Bcryptjs. Integrated to conform to "industry good practices" by ensuring password hashing for all members.
- **Client Application**: Flutter Mobile App. Developed to consume the custom API, providing a cross-platform interface for citizens and officers.
- **Testing**: Postman. Used to verify that "Source code runs with test data" and that "Data is inserted, edited, and retrieved" correctly.

Industry Standards & Literature Review

To ensure the SLBFE system meets modern professional benchmarks, the development followed established industry frameworks and academic principles of web architecture.

- **Security Standard (Bcrypt & OWASP)**:

  The **OWASP (Open Web Application Security Project)** recommends one-way adaptive hashing for credential storage to prevent data breaches. **Bcrypt** was selected as it implements the Eksblowfish algorithm, which includes a "work factor" (salt) that makes it computationally resistant to brute-force and rainbow table attacks.

- **Architectural Style (REST)**:

  The system utilizes **Representational State Transfer (REST)** architecture, as defined by Roy Fielding. This ensures the API is stateless, scalable, and provides a uniform interface that allows different clients (like the Flutter mobile app) to interact with the backend seamlessly using standard HTTP verbs.

- **Data Interchange (JSON)**:

  JSON (JavaScript Object Notation) is utilized as the primary data format because it is language-independent and offers a smaller footprint than XML, making it the industry standard for mobile-to-cloud communication where bandwidth may be limited.

API Documentation

This section provides a comprehensive breakdown of the RESTful interfaces developed for the SLBFE portal. Each route is designed to handle **JSON data** and follows **REST architecture** to ensure industry-standard interoperability.

Core Functional APIs

1. Member Registration

- **Route:** POST /citizens
- **Method**: POST
- **Description**: Facilitates the free online registration for any citizen or officer to become a member. It captures mandatory personal details, including National ID, current GPS coordinates, and a secure hashed password.
- **Parameters**: nid, name, age, address, location (lat/long), profession, affiliation, password, emergencyContact

```
46
47  // 1. MEMBER REGISTRATION (Requirement i)
48  // Citizens and officers can register themselves securely [cite: 12, 23, 24]
49  app.post('/citizens', async (req, res) => {
50      try {
51          if (req.body.nid) req.body.nid = req.body.nid.toString().trim();
52
53          // --- SECURITY: PASSWORD HASHING ---
54          // Conforming to Industry Good Practice for data protection
55          const salt = await bcrypt.genSalt(10);
56          req.body.password = await bcrypt.hash(req.body.password, salt);
57
58          const newCitizen = new Citizen(req.body);
59          await newCitizen.save();
60          res.status(201).json({ message: "Registration successful" });
61      } catch (err) { res.status(400).json({ error: err.message }); }
62  });
```

Figure 1: Code snippet for POST /citizens route

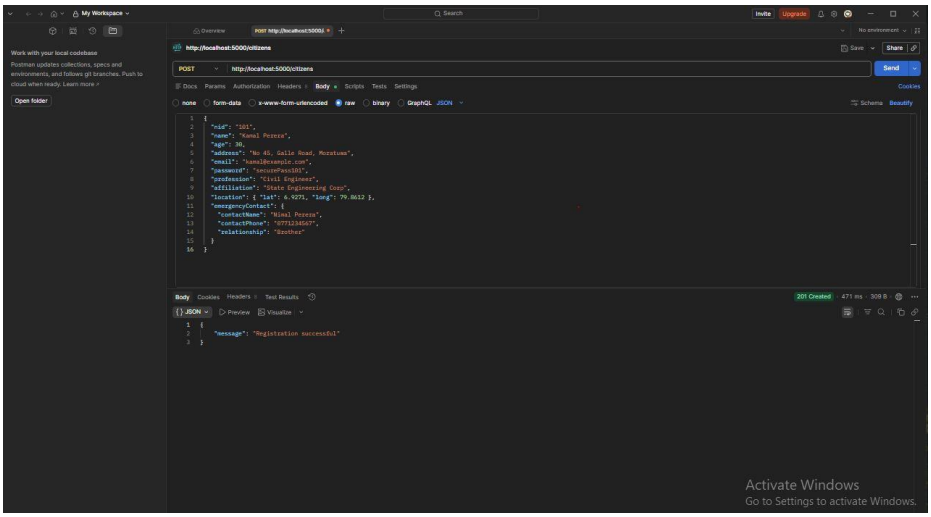

Figure 2: Postman Run-time Result (201 Created) for Kamal Perera

2. Job Qualification & Document Upload

- **Route:** PUT /citizens/:nid/documents
- **Method**: PUT
- **Description**: Allows job seekers to update their professional qualifications and upload essential legal documents such as CVs and birth certificates.
- **Parameters**: nid (URL), qualifications (Body), cv (File), birthCertificate (File) , passportCopy (File)

```
app.put('/citizens/:nid/documents', upload.fields([
    { name: 'birthCertificate', maxCount: 1 },
    { name: 'cv', maxCount: 1 },
    { name: 'passportCopy', maxCount: 1 }
]), async (req, res) => {
    try {
        const searchNid = req.params.nid.trim();
        const updateData = { $set: {} };

        if (req.body.qualifications && req.body.qualifications !== "") {
            updateData.$push = { qualifications: req.body.qualifications };
        }

        if (req.files) {
            if (req.files['birthCertificate']) updateData.$set.birthCertificate = req.files['birthCertificate'][0].path;
            if (req.files['cv']) updateData.$set.cv = req.files['cv'][0].path;
            if (req.files['passportCopy']) updateData.$set.passportCopy = req.files['passportCopy'][0].path;
        }

        const updated = await Citizen.findOneAndUpdate(
            getNidQuery(searchNid),
            updateData,
            { new: true }
        );

        if (!updated) return res.status(404).json({ message: "Citizen record not found" });
        res.json({ message: "Update successful", data: updated });
    } catch (err) { res.status(500).json({ error: err.message }); }
});
```

Figure 3: PUT route code

```
if (!fs.existsSync('./uploads')) {
    fs.mkdirSync('./uploads');
}

const storage = multer.diskStorage({
    destination: (req, file, cb) => {
        cb(null, './uploads');
    },
    filename: (req, file, cb) => {
        // Industry practice: Prevent overwriting with unique timestamps
        cb(null, `${req.params.nid}-${file.fieldname}-${Date.now()}${path.extname(file.originalname)}`);
    }
});
```

Figure 4: Multer configuration

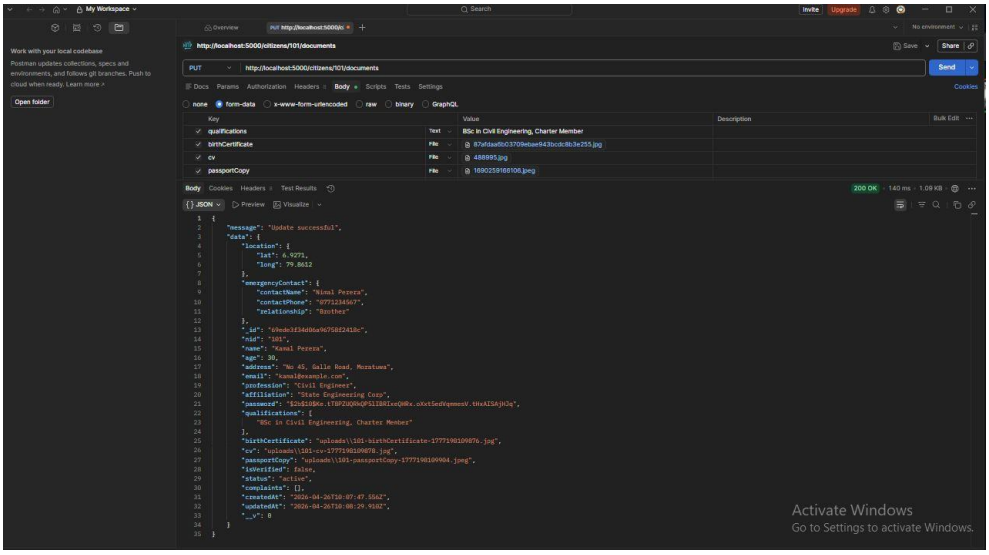

Figure 5: Postman result showing updated qualification string

3. Individual Member Retrieval

- **Route:** GET /citizens/:nid
- **Method**: GET
- **Description**: Enables bureau officers to access a comprehensive profile of any registered member using their unique National ID.
- **Parameters**: nid (URL).

```
app.get('/citizens/:nid', async (req, res) => {
    try {
        const citizen = await Citizen.findOne(getNidQuery(req.params.nid));
        if (!citizen) return res.status(404).json({ message: "Citizen not found" });
        res.json(citizen);
    } catch (err) { res.status(500).json({ error: err.message }); }
});
```

Figure 6: GET /citizens/:nid code snippet

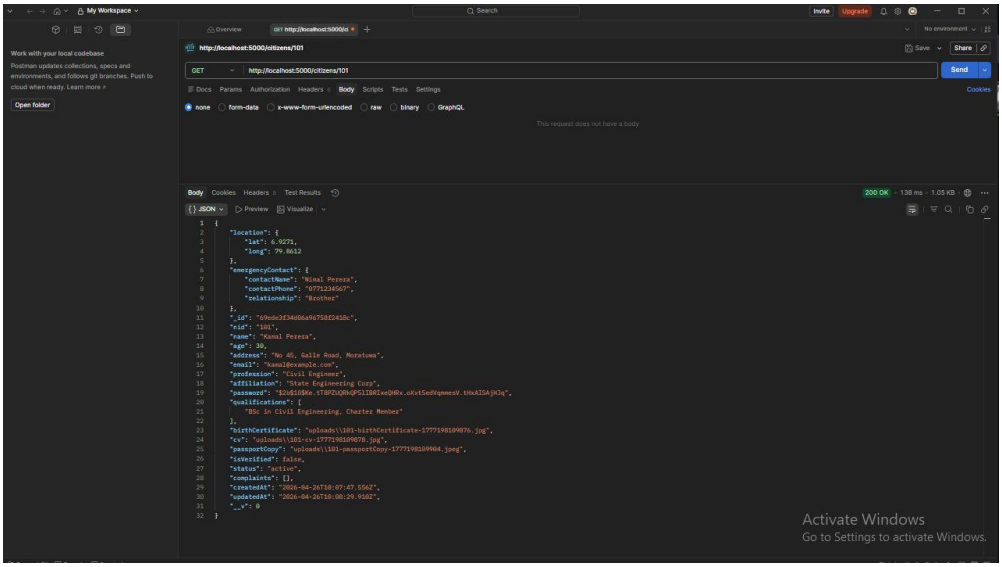

Figure 7: Postman result showing Kamal's full JSON profile

4. Officer Verification of Information

- **Route:** PATCH /citizens/:nid/verify
- **Method**: PATCH
- **Description**: Facilitates the official validation of job seeker data by updating the isVerified status. A PATCH method was selected over PUT to follow RESTful industry standards for partial resource updates, ensuring higher data integrity as per **functional requirement (iv).**
- **Parameters**: nid (URL).

```
app.patch('/citizens/:nid/verify', async (req, res) => {
    try {
        const updated = await Citizen.findOneAndUpdate(
            getNidQuery(req.params.nid),
            { isVerified: true },
            { new: true }
        );
        if (!updated) return res.status(404).json({ message: "NID not found" });
        res.json({ message: "Verified successfully", data: updated });
    } catch (err) { res.status(500).json({ error: err.message }); }
});
```

Figure 8: Code snippet for verification PATCH route

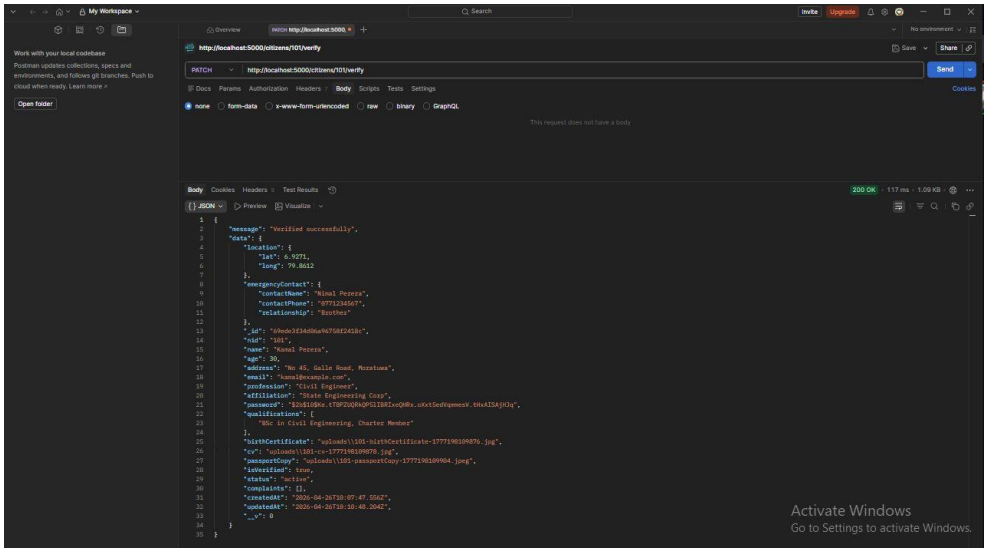

Figure 9: Postman result showing "isVerified: true"

5. Qualification-Based Candidate Search

- **Route:** GET /citizens/find/search
- **Method**: GET
- **Description**: Provides a search interface for foreign companies to "find workers based on the qualifications". It uses a query parameter to filter the database.
- **Parameters**: qualification (Query string).

```
app.get('/citizens/find/search', async (req, res) => {
    try {
        const { qualification } = req.query;
        const workers = await Citizen.find({
            qualifications: { $regex: new RegExp(qualification, "i") },
            isVerified: true,
            status: 'active'
        });
        res.json(workers);
    } catch (err) { res.status(500).json({ error: err.message }); }
});
```

Figure 10: Code for finding candidates by qualification

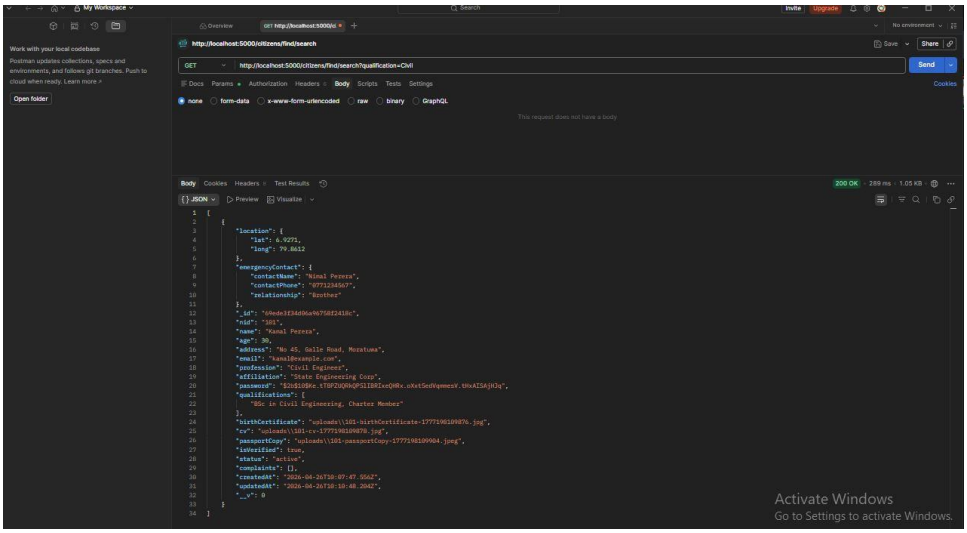

Figure 11: Postman result showing filtered worker list

6. Account Deactivation (Deceased Records)

- **Route:** DELETE /citizens/:nid
- **Method**: DELETE
- **Description**: Provides SLBFE staff the ability to deactivate an individual's account in the event the citizen is deceased, ensuring database integrity
- **Parameters**: nid (URL).

```
app.delete('/citizens/:nid', async (req, res) => {
    try {
        const searchNid = req.params.nid.trim();
        const updated = await Citizen.findOneAndUpdate(
            getNidQuery(searchNid),
            { status: 'deactivated' },
            { new: true }
        );
        if (!updated) return res.status(404).json({ message: "NID not found" });
        res.json({ message: "Account successfully deactivated", data: updated });
    } catch (err) { res.status(500).json({ error: err.message }); }
});
```

Figure 12: DELETE route code snippet

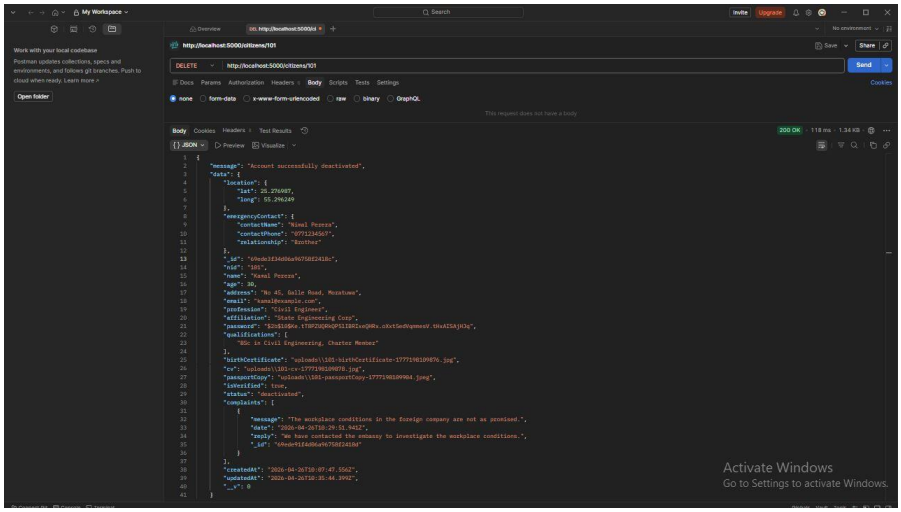

Figure 13: Postman Result: Successful Account Deactivation

7. Emergency Contact Retrieval

- **Route:** GET /citizens/:nid/contacts
- **Method**: GET
- **Description**: Allows SLBFE staff to quickly "collect information about contacts of any citizen" for emergency or monitoring purposes.
- **Parameters**: nid (URL).

```
app.get('/citizens/:nid/contacts', async (req, res) => {
    try {
        const searchNid = req.params.nid.trim();
        const citizen = await Citizen.findOne(getNidQuery(searchNid)).select('emergencyContact name');
        if (!citizen) return res.status(404).json({ message: "NID not found" });
        res.json(citizen.emergencyContact);
    } catch (err) { res.status(500).json({ error: err.message }); }
});
```

Figure 14: GET contacts route code snippet

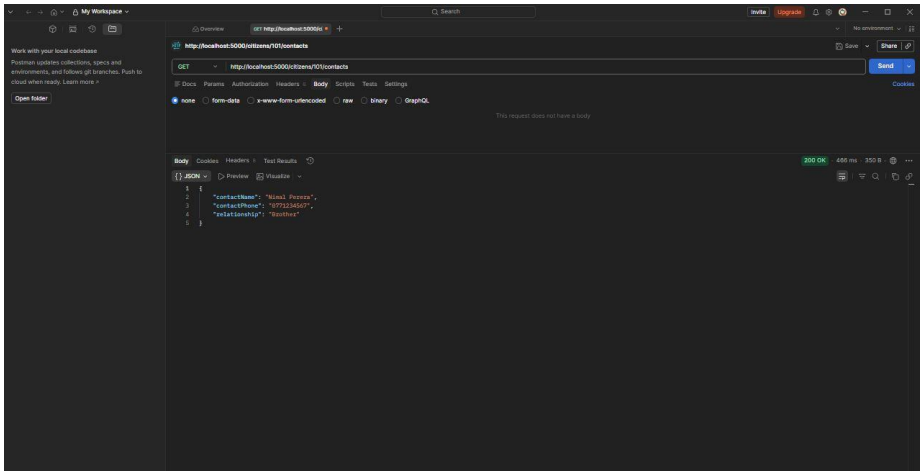

Figure 15: Postman result showing emergency contact JSON

Supplementary Suggested & Implemented APIs

To further satisfy the scenario items and conform to industry good practice, the following five APIs were suggested and implemented.

8. Secure Authentication (Login)

- **Route:** POST /citizens/login
- **Method**: POST
- **Description**: Implements a secure entry point for the portal. It uses **bcrypt.compare** to verify hashed passwords, returning the user's "role" (Officer vs. Citizen) to the mobile client.
- **Parameters**: nid, password.

```
app.post('/citizens/login', async (req, res) => {
    try {
        const { nid, password } = req.body;
        const citizen = await Citizen.findOne(getNidQuery(nid));

        if (!citizen) return res.status(404).json({ message: "NID not registered" });

        // --- SECURITY: BCRYPT COMPARISON ---
        const isMatch = await bcrypt.compare(password, citizen.password);
        if (!isMatch) {
            return res.status(401).json({ message: "Invalid credentials" });
        }

        // Determine role based on 'affiliation' (SLBFE = Officer) [cite: 14, 24]
        const role = citizen.affiliation?.toUpperCase() === 'SLBFE' ? 'officer' : 'citizen';

        res.json({
            message: "Login successful",
            role: role,
            name: citizen.name,
            nid: citizen.nid
        });
    } catch (err) { res.status(500).json({ error: err.message }); }
});
```

Figure 16: Bcrypt Comparison Logic for Secure Login

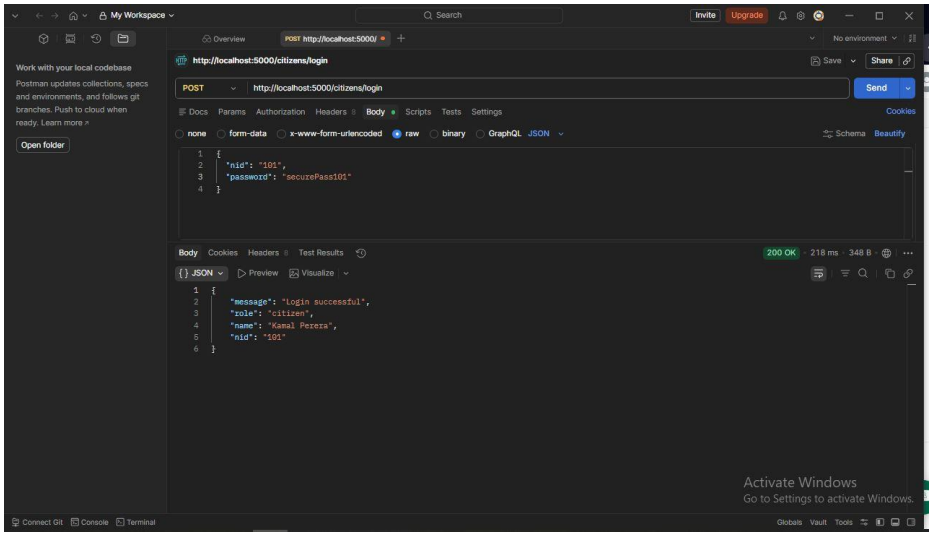

Figure 17: Postman Result: Successful Login with Role Identification

9. Foreign Location Monitoring

- **Route:** PUT /citizens/:nid/location
- **Method**: PUT
- **Description**: Specifically addresses Scenario **Item (v)**. It allows citizens who have traveled abroad to "update their current location as soon as they visit the foreign company"
- **Parameters**: nid (URL), lat (Body), long (Body)

```
app.put('/citizens/:nid/location', async (req, res) => {
    try {
        const { lat, long } = req.body;
        const updated = await Citizen.findOneAndUpdate(
            getNidQuery(req.params.nid),
            { $set: { "location.lat": lat, "location.long": long } },
            { new: true }
        );
        if (!updated) return res.status(404).json({ message: "NID not found" });
        res.json(updated);
    } catch (err) { res.status(500).json({ error: err.message }); }
});
```

Figure 18: Update Coordinates Logic

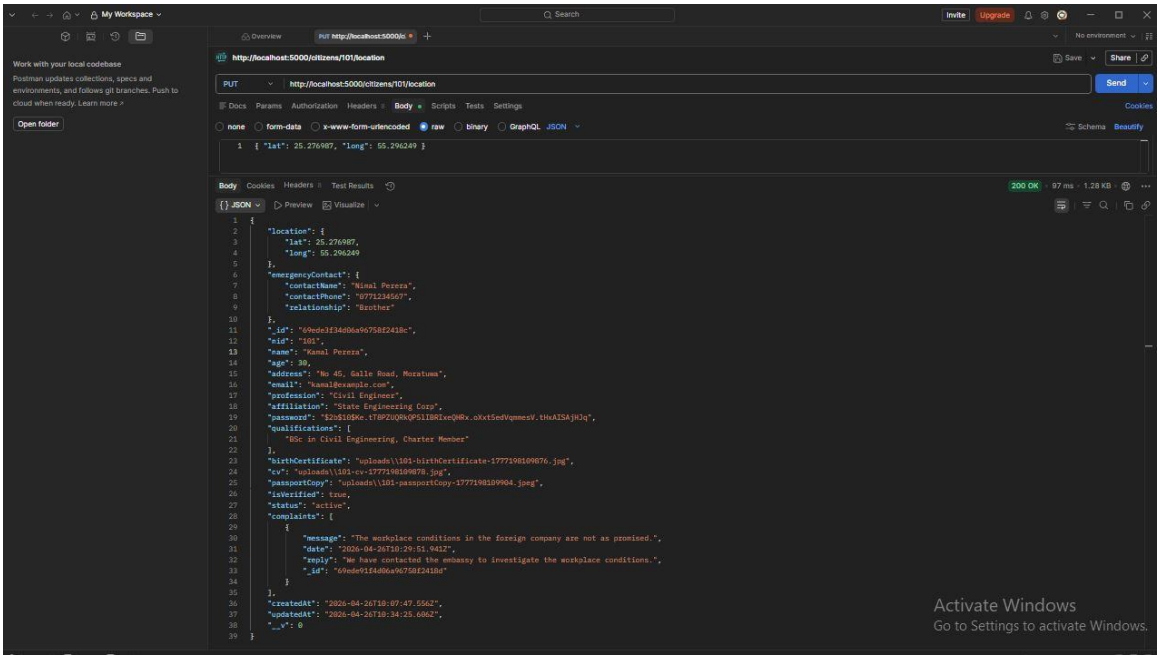

Figure 19: Postman Result: Updated GPS Coordinates for Overseas Monitoring

10. Complaint Lodging System

- **Route:** POST /citizens/:nid/complaint
- **Method**: POST
- **Description**: Addresses Scenario **Item (vi)**, enabling any citizen to lodge a formal complaint regarding their foreign employment conditions.
- **Parameters**: nid (URL), message (Body).

```
app.post('/citizens/:nid/complaint', async (req, res) => {
    try {
        const { message } = req.body;
        const updated = await Citizen.findOneAndUpdate(
            getNidQuery(req.params.nid),
            { $push: { complaints: { message, date: new Date(), reply: "" } } },
            { new: true }
        );
        if (!updated) return res.status(404).json({ message: "NID not found" });
        res.json(updated);
    } catch (err) { res.status(500).json({ error: err.message }); }
});
```

Figure 20: Citizen Complaint Submission Logic

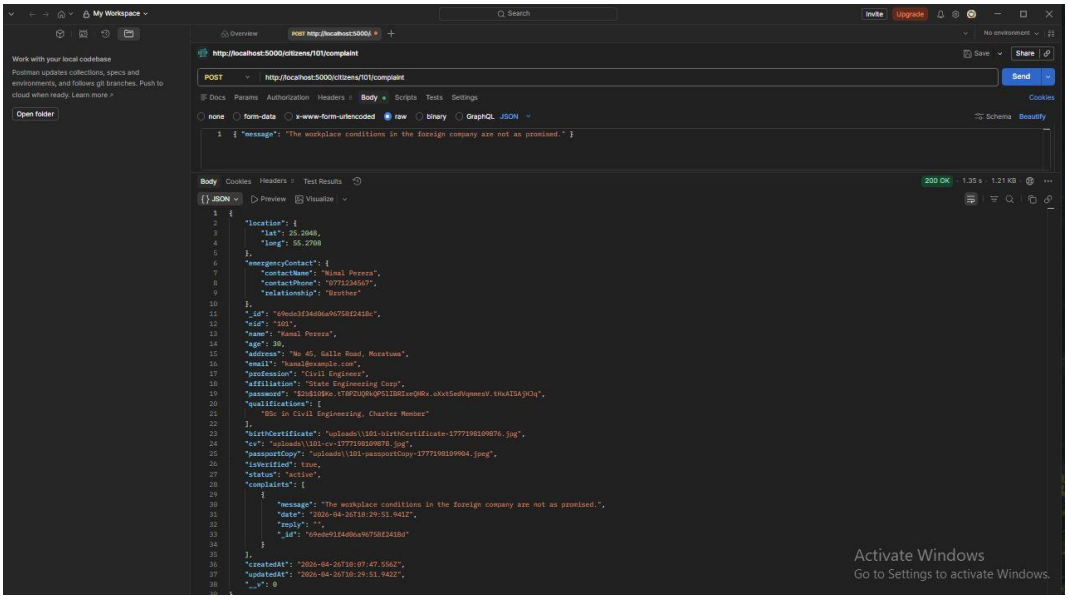

Figure 21: Postman Result: Successful Complaint Entry in Member Profile

11. Bureau Complaint Resolution

- **Route:** PATCH /citizens/:nid/complaint/:id
- **Method**: PATCH
- **Description**: Satisfies the requirement for bureau officers to "see the content and reply accordingly" to citizen complaints.
- **Parameters**: nid (URL), complaint id (URL), reply (Body).

```
app.patch('/citizens/:nid/complaint/:complaintId', async (req, res) => {
    try {
        const updated = await Citizen.findOneAndUpdate(
            { ...getNidQuery(req.params.nid), "complaints._id": req.params.complaintId },
            { $set: { "complaints.$.reply": req.body.reply } },
            { new: true }
        );
        if (!updated) return res.status(404).json({ message: "Complaint ID not found" });
        res.json(updated);
    } catch (err) { res.status(500).json({ error: err.message }); }
});
```

Figure 22: Officer Reply Logic for Grievance Handling

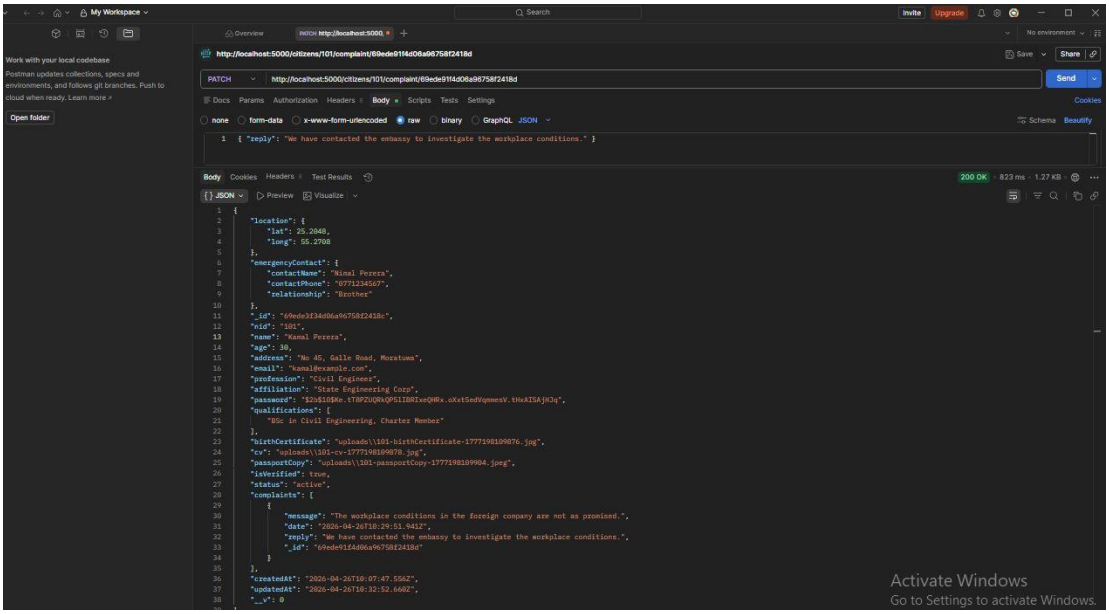

Figure 23: Postman Result: Verified Official Reply Saved in Database

12. Administrative Dashboard (All Members)

- **Route:** GET /citizens
- **Method**: GET
- **Description**: Provides a collection view of all registered members. This is used by the mobile client's **Officer Dashboard** to show a high-level overview of the bureau's data.
- **Parameters**: None.

```
app.get('/citizens', async (req, res) => {
    try {
        const citizens = await Citizen.find();
        res.json(citizens);
    } catch (err) { res.status(500).json({ error: err.message }); }
});
```

Figure 24: Global Collection Retrieval Logic

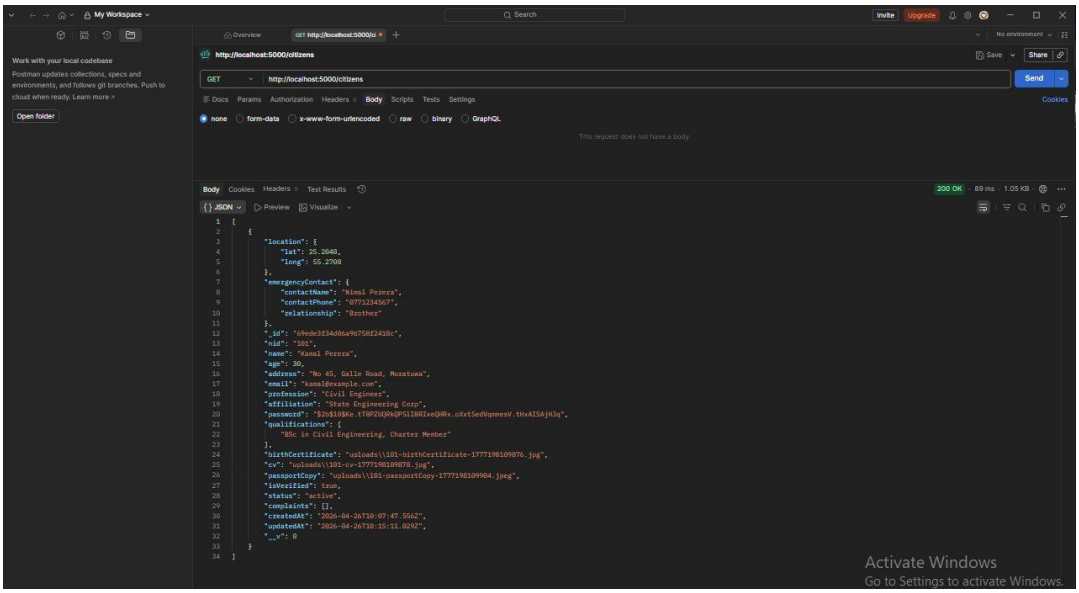

Figure 25: Postman Result: Full Registry View for Officer Dashboard

Industry Good Practices & Security

- **Security**: Passwords are never stored in plain text. Instead, **bcrypt hashing** was used to ensure user privacy.

- **RESTful Architecture**: Every interface uses proper HTTP verbs (POST, GET, PUT, PATCH, DELETE) and delivers data strictly in **JSON format**.

- **Semantic API Design (Domain-Driven Naming)**: API endpoints follow domain-specific naming conventions aligned with the SLBFE business context (e.g., /citizens instead of /users). This conforms to REST API design best practices as recommended by industry standards, improving maintainability and interoperability across different client applications.

Client Application Description

The developed Flutter mobile application acts as the primary interface for the SLBFE system, catering to three distinct user groups: Citizens, Foreign Companies, and Bureau Officers.

1. Access & Core Navigation

- The initial gateway of the system where users enter their National ID and password. This screen facilitates the POST /citizens/login request, verifying hashed credentials against the MongoDB database to determine the user's role and permissions. It also provides a clear navigation link for unregistered citizens to join the bureau.

- The entry point after successful authentication. It provides a role-based menu system and features a personalized welcome message powered by the login response, ensuring a professional user experience (UX).

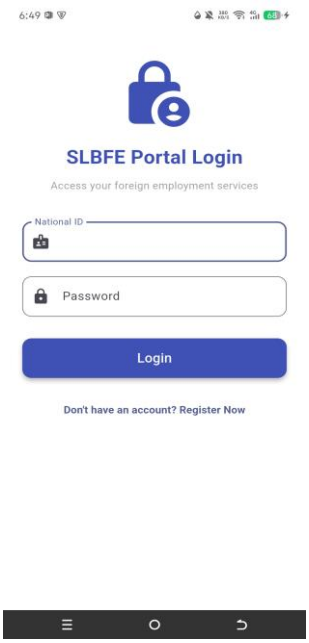

Figure 26: Secure Login Interface

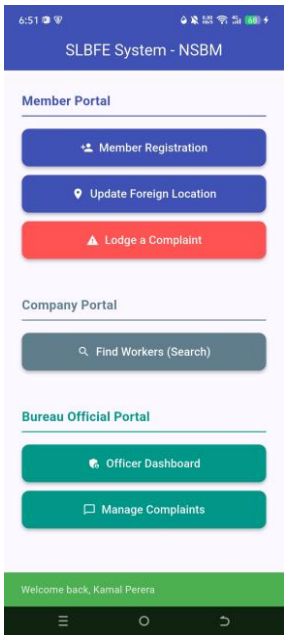

Figure 27: Central Navigation Dashboard

2. Member Hub (Public Interface)

Member Registration & GPS Capture

Satisfies Requirement (i) by capturing personal details and the initial registration location coordinates.

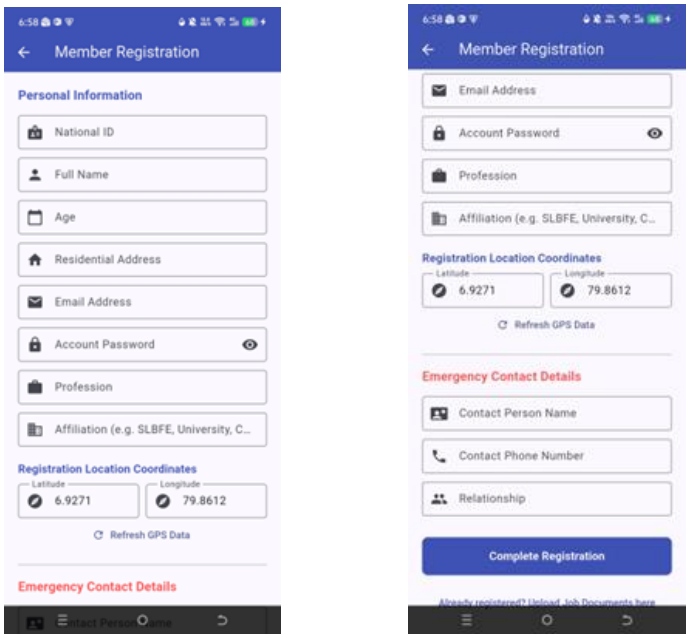

Figure 28: Member Registration UI

Job Seeker Portal (Qualifications & Documents)

Directly addresses Requirement (ii), allowing users to update professional skills and upload digital copies of birth certificates, CVs, and passports.

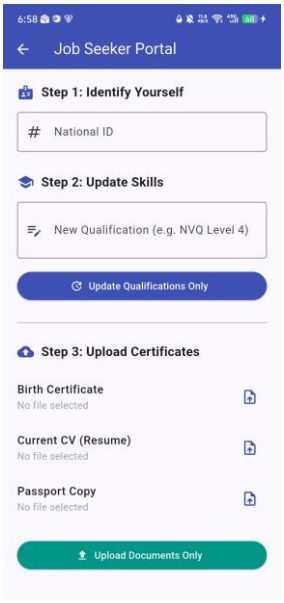

Figure 29: Job Seeker Portal

Foreign Location Monitoring Interface

Facilitates Requirement (v) for citizens to update their specific coordinates upon visiting a foreign company.

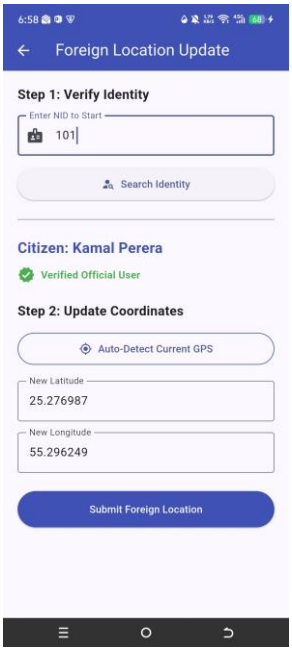

Figure 30: Foreign Location Update Screen

Citizen Grievance & Complaint Lodge

Enables any citizen to formally lodge a complaint regarding their workplace or family issues as per Requirement (vi).

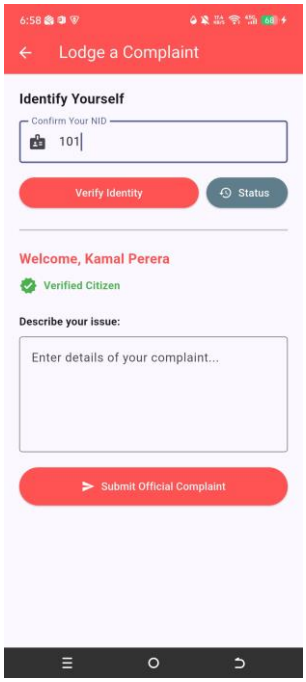

Figure 31: Citizen Grievance Lodge Interface

Complaint Status & Official Replies

Provides a transparent view for citizens to see the bureau's response to their submitted complaints.

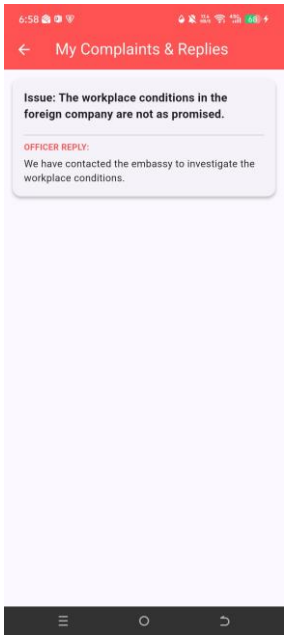

Figure 32: Complaint Status & Official Replies View

3. Company Portal (Recruiter Interface)

Worker Discovery & Qualification Search

Directly fulfilling Scenario Item (iv) and Functional Requirement (v), this interface provides foreign recruitment firms with a specialized search tool to discover competent workers based on specific qualifications, ensuring the bureau meets the global human resource needs of overseas markets.

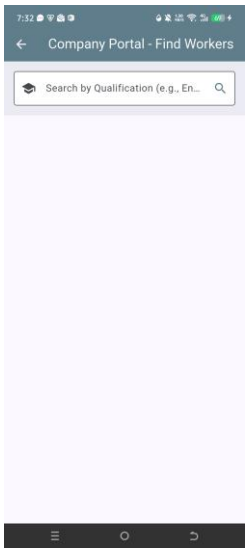

Figure 33: Recruiter Search Portal

4. Bureau Officer Portal (Administrative Interface)

This secure portal provides the SLBFE staff with the tools needed to monitor and validate the database.

Bureau Officer Dashboard & Member Status

Shows a collection view of members, including the ability to identify "DEACTIVATED" accounts for deceased individuals.

Figure 34: Bureau Officer Dashboard

Emergency Contact Retrieval

A specialized view for staff to "collect information about contacts of any citizen" as required by functional requirement (vii).

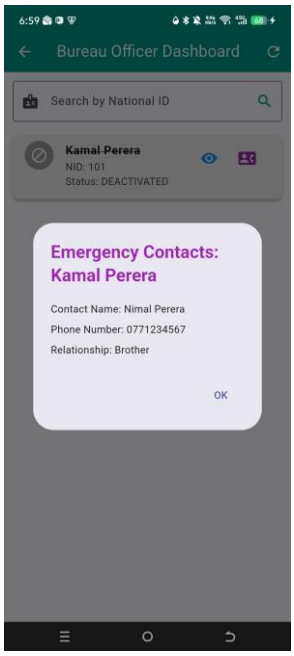

Figure 35: Emergency Contact Retrieval Interface

Comprehensive Member Profile Audit

The full detailed view enabling officers to "see and validate information" provided by seekers, including document verification (Requirement iii).

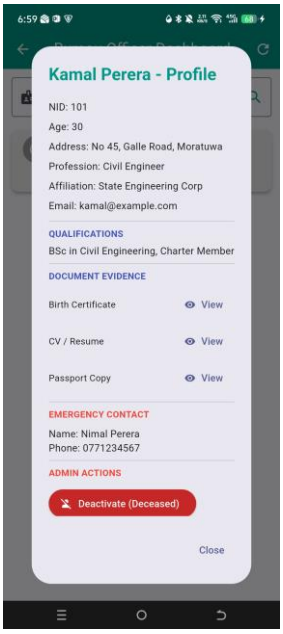

Figure 36: Comprehensive Member Audit & Document View

Official Complaint Resolution Hub

The interface used by officers to "see the content and reply accordingly" to worker grievances.

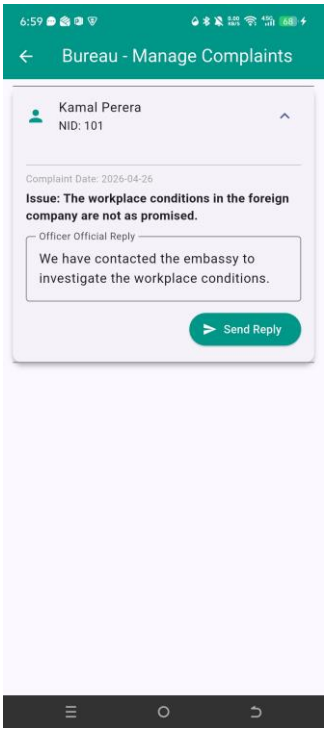

Figure 37: Complaint Resolution Hub

Individual Contributions

As the sole developer of this project, my contributions included:

- **System Design**: Analyzing the SLBFE scenario and designing the RESTful API endpoints.
- **Backend Development**: Implementing the Node.js server, MongoDB database connectivity, and bcrypt security measures.
- **Mobile Development**: Building the Flutter client application and ensuring successful integration with the backend API.
- **Quality Assurance**: Conducting rigorous API testing using Postman to ensure all functional requirements were met.

Conclusion

This project successfully demonstrates the implementation of a modern, secure, and robust web API for the SLBFE. By adhering to RESTful principles and industry good practices, the system provides a scalable solution for managing human resources in the overseas market while satisfying all coursework requirements.